

Database Management Systems

A friendly first course in databases, SQL and
transactions

Sample study notes · Knowledge Dungeon

Contents

Introduction to Databases	3
Data Models and the ER Model	13
The Relational Model and Keys	23
Relational Algebra	32
SQL Basics	41
Advanced SQL: Joins, Grouping and Subqueries	49
Functional Dependencies and Normalization	59
Transactions and the ACID Properties	68
Concurrency Control	77
Storage, Indexing and Recovery	86

Introduction to Databases

Almost every app you use stores data somewhere. Your college portal remembers your marks, a food delivery app remembers your address, and a bank remembers every rupee that moves through your account. This chapter explains what a database is, why we need special software to manage it, and the vocabulary you will use for the rest of this book.

What Is Data?

Data is a collection of raw facts: a name, a roll number, a price, a date. On its own, a single fact rarely means much. The number 82 could be a mark, a temperature or a page count. Data becomes **information** when it is organised and given context: "Riya scored 82 in DBMS" is information.

A **database** is an organised collection of related data, stored so that it can be searched, updated and shared easily. The key word is *organised*. A pile of notebooks full of marks is data; a well-structured table of students, subjects and marks is a database.

Key idea: A database is not just storage. It is storage with structure, so that questions like "which students scored above 80 in DBMS?" can be answered quickly and correctly.

Life Before Databases: The File System Approach

Before database systems became common, organisations stored data in ordinary files. Each department wrote its own programs and kept its own files. The accounts office had a fees file, the exam cell had a marks file and the library had a borrowers file.

This worked for small systems, but it created serious problems as systems grew.

Problems with File-Based Systems

1. **Data redundancy.** The same student's name and address were copied into many files. Storage was wasted and the copies easily went out of sync.
2. **Data inconsistency.** When a student changed their phone number, one office updated its file and another did not. Now the organisation had two different "truths".
3. **Difficult access.** Every new question needed a new program. If the principal wanted "all hostel students with pending fees", someone had to write fresh code to combine two files.
4. **Data isolation.** Files were stored in different formats by different programs, so combining them was painful.

5. **Integrity problems.** Rules such as "marks must be between 0 and 100" were buried inside program code. Adding or changing a rule meant changing every program.
6. **Atomicity problems.** If the power failed halfway through transferring money from one account to another, one file might be updated and the other not. Money could vanish.
7. **Concurrent access anomalies.** If two clerks updated the same file at the same time, one person's changes could overwrite the other's.
8. **Security problems.** It was hard to allow the exam cell to see marks while hiding fee details from them.

The Database Management System

A **Database Management System (DBMS)** is software that stores data in a database and provides tools to define, insert, query, update and protect it. Instead of every program reading raw files, programs ask the DBMS for data, and the DBMS takes care of the details.

Popular examples include MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server and SQLite. SQLite is small enough to live inside a mobile app, while Oracle can run the systems of a large bank.

What a DBMS Gives You

Problem with files	How a DBMS helps
Redundancy and inconsistency	Data is stored once and shared by all programs
Difficult access	A query language (SQL) answers new questions without new programs
Integrity rules scattered in code	Constraints are declared once, inside the database
Partial updates after failures	Transactions are all-or-nothing
Clashing concurrent users	Concurrency control keeps simultaneous work correct
Weak security	Users and roles get exactly the permissions they need

Levels of Abstraction

A DBMS hides complexity by describing data at three levels. This is called the **three-schema architecture**.

- **Physical level.** How data is actually stored on disk: files, blocks, indexes and compression. Only database administrators and the DBMS itself care about this level.
- **Logical (conceptual) level.** What data is stored and how it is related: the tables, their columns and the rules between them. Developers mostly work here.
- **View (external) level.** What a particular user is allowed to see. A student might see only their own marks; a teacher might see marks for their subject only.

Key idea: Each level hides the details of the level below it. A student viewing their marks does not need to know which disk block holds them.

Data Independence

Because the levels are separate, we can change one level without breaking the others.

- **Physical data independence** means we can change how data is stored (for example, add an index or move to a faster disk) without changing the logical schema or application programs.
- **Logical data independence** means we can change the logical schema (for example, add a new column) without breaking existing views and programs.

Physical independence is easy to achieve and very common.

Logical independence is harder, because programs often depend closely on table structure.

Schema and Instance

The **schema** is the design of the database: the names of tables, their columns and their types. It changes rarely. The **instance** (or *state*) is the actual data stored at a particular moment. It changes all the time.

Think of a schema as the shape of an empty form and the instance as all the filled-in copies of that form today.

```
Schema:  STUDENT(roll_no, name, branch, year)
```

```
Instance: (101, 'Riya', 'CSE', 2)
          (102, 'Arjun', 'IT', 2)
          (103, 'Meera', 'CSE', 3)
```

Database Languages

A DBMS provides languages for different jobs. In practice they are all part of **SQL** (Structured Query Language), which you will study in detail in Chapter 5.

- **DDL (Data Definition Language)** defines the structure:
CREATE TABLE, ALTER TABLE, DROP TABLE.
- **DML (Data Manipulation Language)** works with the data itself: SELECT, INSERT, UPDATE, DELETE.
- **DCL (Data Control Language)** manages permissions:
GRANT, REVOKE.
- **TCL (Transaction Control Language)** manages transactions: COMMIT, ROLLBACK, SAVEPOINT.

```
CREATE TABLE student (  
    roll_no INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    branch VARCHAR(10),  
    year INT CHECK (year BETWEEN 1 AND 4)  
);  
  
INSERT INTO student VALUES (101, 'Riya', 'CSE', 2);  
  
SELECT name FROM student WHERE branch = 'CSE';
```

Who Uses a Database?

- **End users** use applications built on the database, such as a student checking results online.
- **Application programmers** write the programs that talk to the database.
- **Database designers** decide which tables exist and how they relate.
- **Database administrators (DBAs)** install and tune the DBMS, manage backups, grant permissions and keep the system healthy.

DBMS Architecture

Most real applications today use a **three-tier architecture**:

1. The **presentation tier** is what the user sees: a website or mobile app.

2. The **application tier** holds business logic, for example the rules for calculating a CGPA.
3. The **data tier** is the DBMS, which stores and protects the data.

The user never talks to the database directly. This keeps the database safer and lets each tier be changed independently. Older systems sometimes used a **two-tier** design, in which a desktop program connected straight to the database.

Advantages and Disadvantages of a DBMS

Advantages:

- Controlled redundancy and consistent data
- Easy sharing between many users and programs
- Strong security and integrity rules
- Backup and recovery after crashes
- Powerful query languages

Disadvantages:

- The software can be expensive and complex
- It needs skilled people to run it well
- For a tiny, single-user program, a DBMS can be more than you need

Summary

- Data is raw facts; information is organised data with meaning.
- File-based systems suffer from redundancy, inconsistency, difficult access, integrity, atomicity, concurrency and security problems.
- A DBMS stores data centrally and provides querying, constraints, transactions, concurrency control and security.
- The three-schema architecture separates physical, logical and view levels, giving physical and logical data independence.
- A schema is the design; an instance is the data at one moment.
- SQL includes DDL, DML, DCL and TCL commands.

Practice Questions

1. List four problems of file-based systems and explain how a DBMS solves each.
2. What is the difference between a schema and an instance? Give an example.
3. Explain physical and logical data independence. Which is harder to achieve, and why?

4. Classify each command as DDL, DML, DCL or TCL: `SELECT`, `GRANT`, `ALTER`, `COMMIT`, `DELETE`.
5. Draw the three-tier architecture for an online exam portal and label each tier.

Data Models and the ER Model

Before we build tables, we need a plan. Just as an architect draws a blueprint before construction starts, a database designer draws a model of the data before writing any SQL. This chapter introduces data models and the most popular design tool of all: the Entity-Relationship (ER) model.

What Is a Data Model?

A **data model** is a set of concepts used to describe the structure of a database, the relationships between pieces of data and the rules the data must follow. Different data models describe data in different ways.

- **Hierarchical model.** Data is arranged as a tree, with each child having exactly one parent. It was used in early IBM systems. It is fast for fixed queries but awkward when one record belongs to two parents.
- **Network model.** Like the hierarchical model, but a record can have many parents, forming a graph. It is flexible but complicated to program.
- **Relational model.** Data is stored in tables (relations). It is simple, mathematically sound and by far the most widely used today. Chapter 3 covers it in depth.

- **Object-oriented model.** Data is stored as objects with attributes and methods, matching object-oriented programming languages.
- **Document and key-value models.** Used by NoSQL databases such as MongoDB and Redis. Data is stored as flexible JSON-like documents or simple key-value pairs.

Key idea: The ER model is a *conceptual* model. We use it to think and communicate. Afterwards we convert the ER diagram into relational tables.

The Design Process

Designing a database usually follows these steps:

1. **Requirements analysis.** Talk to users and find out what data must be stored and what questions must be answered.
2. **Conceptual design.** Draw an ER diagram describing entities, attributes and relationships.
3. **Logical design.** Convert the ER diagram into tables (the relational schema).
4. **Schema refinement.** Improve the tables using normalization (Chapter 7).
5. **Physical design.** Choose indexes and storage options for performance.

Entities and Entity Sets

An **entity** is a real-world thing that can be distinctly identified: a particular student, a particular course, a particular book. An **entity set** is a collection of similar entities, such as all students.

In an ER diagram, an entity set is drawn as a **rectangle**.

Strong and Weak Entities

A **strong entity** has an attribute (or set of attributes) that uniquely identifies each entity. Every student has a unique roll number, so STUDENT is a strong entity.

A **weak entity** cannot be identified by its own attributes alone. Consider a DEPENDENT of an employee, such as a child. Two different employees might both have a child named "Aarav". The dependent is identified only by combining the employee's ID with the dependent's name. Weak entities are drawn as **double rectangles**, and the attribute that partly identifies them (such as the name) is called a **partial key** or **discriminator**.

Attributes

An **attribute** is a property of an entity, such as a student's name or date of birth. Attributes are drawn as **ovals** connected to their entity.

Attribute type	Meaning	Example
Simple	Cannot be divided further	Roll number

Attribute type	Meaning	Example
Composite	Made of smaller parts	Name (first, middle, last)
Single-valued	One value per entity	Date of birth
Multivalued	Many values per entity	Phone numbers
Stored	Stored directly	Date of birth
Derived	Calculated from other attributes	Age (from date of birth)
Key	Uniquely identifies an entity	Roll number

In diagrams, a multivalued attribute has a **double oval**, a derived attribute has a **dashed oval**, and a key attribute is **underlined**.

Relationships

A **relationship** is an association between entities. "Riya *enrolls* in DBMS" is a relationship between a student and a course. A **relationship set** is a collection of similar relationships. Relationship sets are drawn as **diamonds**.

The **degree** of a relationship is the number of entity sets taking part.

- **Unary (recursive):** one entity set related to itself, such as EMPLOYEE *manages* EMPLOYEE.
- **Binary:** two entity sets, such as STUDENT *enrolls in* COURSE. This is the most common.

- **Ternary:** three entity sets, such as SUPPLIER *supplies* PART to PROJECT.

Relationships can have attributes too. The *grade* a student earns belongs to the enrolment, not to the student alone or the course alone, so it is an attribute of the ENROLLS relationship.

Cardinality Ratios

Cardinality tells us how many entities on one side can be related to entities on the other side.

- **One-to-one (1:1).** Each person has at most one passport, and each passport belongs to one person.
- **One-to-many (1:N).** One department has many employees, but each employee works in one department.
- **Many-to-one (N:1).** The same as one-to-many, read from the other side.
- **Many-to-many (M:N).** A student enrolls in many courses, and a course has many students.

Key idea: Getting cardinality right matters, because it decides how the relationship becomes tables later. An M:N relationship always needs its own table.

Participation Constraints

Participation tells us whether every entity must take part in a relationship.

- **Total participation:** every entity must participate. Every loan must belong to some customer. Drawn with a **double line**.
- **Partial participation:** some entities may not participate. Not every customer has a loan. Drawn with a **single line**.

A Worked Example: A College Database

Suppose a college wants to store students, courses, departments and instructors.

- STUDENT (roll_no, name, phone numbers, date of birth)
- COURSE (course_id, title, credits)
- DEPARTMENT (dept_id, dept_name)
- INSTRUCTOR (emp_id, name, salary)

Relationships:

- A student **enrolls in** many courses, and each course has many students (M:N). The relationship has a *grade* attribute.
- An instructor **teaches** many courses; each course is taught by one instructor (1:N).

- Each instructor **belongs to** exactly one department; a department has many instructors (N:1, total participation for instructors).

In the diagram, STUDENT and COURSE are rectangles joined by an ENROLLS diamond, with *grade* drawn as an oval on the diamond. Phone numbers appear as a double oval on STUDENT, and roll_no is underlined.

Extended ER Features

Large designs often need a few more ideas.

Generalization and Specialization

Specialization is top-down: we start from a general entity and split it into more specific ones. PERSON can be specialised into STUDENT and INSTRUCTOR. **Generalization** is bottom-up: we notice that CAR and TRUCK share many attributes and combine them into VEHICLE. Both form an "is-a" hierarchy, in which the lower entities **inherit** the attributes of the higher entity.

Constraints on specialization:

- **Disjoint vs overlapping:** can an entity belong to more than one subclass? A person could be both a student and a teaching assistant (overlapping).
- **Total vs partial:** must every higher-level entity belong to some subclass?

Aggregation

Sometimes we need a relationship *about* a relationship.

Suppose an instructor guides a student on a project, and we want to record which sponsor funds that guidance.

Aggregation treats the relationship "instructor guides student on project" as a single higher-level entity, which can then take part in another relationship.

Converting an ER Diagram to Tables

The rules are mechanical, which is part of what makes ER design so useful.

1. **Strong entity:** make a table with all simple attributes. The key attribute becomes the primary key.
2. **Composite attribute:** store only its parts (first_name, last_name).
3. **Multivalued attribute:** make a separate table containing the entity's key and the value, such as STUDENT_PHONE(roll_no, phone).
4. **Weak entity:** make a table containing its partial key plus the primary key of its owner. Together they form the primary key.
5. **1:N relationship:** add the key of the "one" side as a foreign key in the table of the "many" side.

6. **1:1 relationship:** add a foreign key on either side, preferably the side with total participation.
7. **M:N relationship:** make a new table containing the keys of both entities plus any relationship attributes.

```
STUDENT(roll_no, name, dob)
STUDENT_PHONE(roll_no, phone)
COURSE(course_id, title, credits, emp_id)      -- 1:N teaches
INSTRUCTOR(emp_id, name, salary, dept_id)      -- N:1
    belongs to
DEPARTMENT(dept_id, dept_name)
ENROLLS(roll_no, course_id, grade)            -- M:N enrolls
```

Summary

- A data model describes structure, relationships and rules. The relational model dominates today.
- The ER model uses entities (rectangles), attributes (ovals) and relationships (diamonds).
- Attributes can be simple or composite, single or multivalued, stored or derived.
- Cardinality (1:1, 1:N, M:N) and participation (total or partial) describe how entities relate.
- Weak entities depend on an owner entity for identification.
- Specialization, generalization and aggregation handle more complex designs.

- ER diagrams convert to tables through a fixed set of rules.

Practice Questions

1. Differentiate between a strong and a weak entity with an example of each.
2. Classify these attributes: address, age, email IDs, Aadhaar number.
3. Draw an ER diagram for a library with books, members and loans. Mark cardinality and participation.
4. Why does an M:N relationship need a separate table? Illustrate with STUDENT and COURSE.
5. Explain aggregation with an example of your own.

The Relational Model and Keys

The relational model, proposed by Edgar F. Codd at IBM in 1970, changed databases forever. Its idea is simple: store all data in tables, and describe queries using mathematics rather than pointers and file positions. Almost every database you will meet in industry is relational. This chapter covers the vocabulary of the model and the keys and constraints that keep data correct.

Relations, Tuples and Attributes

In the relational model, data is stored in **relations**, which we usually draw as tables.

Formal term	Everyday term	Meaning
Relation	Table	A set of rows with the same columns
Tuple	Row / record	One entry in the table
Attribute	Column / field	One property, such as name
Domain	Data type	The allowed values of an attribute
Degree	Number of columns	How many attributes a relation has
Cardinality	Number of rows	How many tuples a relation has

For example, the relation STUDENT(roll_no, name, branch, cgpa) has degree 4. If it holds 60 students, its cardinality is 60.

Properties of a Relation

Because a relation is a mathematical *set* of tuples, it has some important properties:

1. **No duplicate tuples.** Two rows cannot be exactly identical.
2. **Order of tuples does not matter.** Rows have no "first" or "last" in theory.
3. **Order of attributes does not matter**, as long as each value stays with its attribute name.
4. **Values are atomic.** Each cell holds a single, indivisible value. A cell cannot contain a list of phone numbers. This is called **First Normal Form**, which we revisit in Chapter 7.

Key idea: Real SQL databases relax some of these rules. SQL tables *can* contain duplicate rows unless you forbid them with a key. This is why keys are so important.

NULL Values

Sometimes a value is unknown or does not apply, such as a middle name for someone who has none. The relational model uses a special marker called **NULL**. NULL is not zero and not an empty string; it means "no value here". Comparisons with NULL behave unusually, which you will see in the SQL chapters.

Relation Schema and Relation Instance

A **relation schema** is written as the relation name followed by its attributes, for example `STUDENT(roll_no, name, branch, cgpa)`.

A **relation instance** is the set of tuples in the relation at a given time. A **relational database schema** is the collection of all relation schemas in the database.

Keys

Keys identify tuples and connect relations together. They are the single most important concept in this chapter.

Super Key

A **super key** is any set of attributes that uniquely identifies a tuple. In `STUDENT`, `{roll_no}` is a super key. So is `{roll_no, name}`, because if `roll_no` alone is unique, adding more attributes keeps it unique. A super key may contain extra, unnecessary attributes.

Candidate Key

A **candidate key** is a *minimal* super key: a super key from which no attribute can be removed without losing uniqueness. If every student has a unique roll number and also a unique email, then both `{roll_no}` and `{email}` are candidate keys. `{roll_no, name}` is a super key but not a candidate key, because `name` is unnecessary.

Primary Key

The **primary key** is the candidate key chosen by the designer to identify tuples. A table has exactly one primary key. Primary key values:

- must be **unique**, and
- must **never be NULL**.

Good primary keys are short, stable and never change. A roll number is a better primary key than a phone number, which might change.

Alternate Key

The candidate keys that were not chosen as the primary key are called **alternate keys**. If roll_no is the primary key, email is an alternate key. We usually still enforce its uniqueness with a UNIQUE constraint.

Composite Key

A key made of more than one attribute is a **composite key**. In ENROLLS(roll_no, course_id, grade), neither roll_no nor course_id alone is unique, because one student takes many courses and one course has many students. Together, {roll_no, course_id} identifies each enrolment.

Foreign Key

A **foreign key** is an attribute in one relation that refers to the primary key of another relation. It creates a link between tables.

```
CREATE TABLE department (
    dept_id   VARCHAR(5) PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL
);

CREATE TABLE student (
    roll_no   INT PRIMARY KEY,
    name      VARCHAR(50) NOT NULL,
    dept_id   VARCHAR(5) REFERENCES department(dept_id)
);
```

Here `student.dept_id` is a foreign key. It may only contain values that exist in `department.dept_id` (or NULL, if allowed). The table containing the foreign key is the **referencing** (child) table; the other is the **referenced** (parent) table.

Surrogate Key

Sometimes no natural attribute makes a good key. A **surrogate key** is an artificial value, usually an auto-incrementing number, created only to identify rows. Many real systems use surrogate keys such as `id SERIAL PRIMARY KEY`.

Integrity Constraints

Constraints are rules that the DBMS enforces automatically, so bad data never enters the database.

Domain Constraints

Each attribute must take values from its domain. A `cgpa` column of type `DECIMAL(4,2)` with `CHECK (cgpa BETWEEN 0 AND 10)` rejects a value of 11.5 or the text "excellent".

Key Constraints

Primary key and unique values must not repeat.

Entity Integrity

No primary key attribute can be NULL. If the primary key could be NULL, we could not identify that row.

Referential Integrity

A foreign key value must either match an existing primary key value in the referenced table or be NULL.

You cannot enrol a student in department "XYZ" if no such department exists.

Referential integrity raises a question: what should happen when a referenced row is deleted or updated? SQL lets you choose.

Option	What happens to child rows when the parent is deleted
RESTRICT / NO ACTION	The delete is refused while children exist
CASCADE	Child rows are deleted too
SET NULL	The foreign key in child rows is set to NULL
SET DEFAULT	The foreign key is set to its default value

```
CREATE TABLE enrolls (  
    roll_no    INT REFERENCES student(roll_no) ON DELETE  
    CASCADE,  
    course_id  VARCHAR(8) REFERENCES course(course_id),  
    grade      CHAR(2),  
    PRIMARY KEY (roll_no, course_id)  
);
```

With ON DELETE CASCADE, deleting a student automatically deletes their enrolments.

Key idea: Constraints move rules out of application code and into the database, where every program must obey them. This solves the integrity problem of file-based systems from Chapter 1.

Codd's Rules in Brief

Codd later published twelve rules (numbered 0 to 12) that a truly relational system should follow. You do not need to memorise them all, but a few capture the spirit of the model:

- **Information rule:** all data is represented as values in tables.
- **Guaranteed access rule:** every value can be reached by table name, primary key and column name.
- **Systematic treatment of NULLs:** NULL is handled consistently, independent of data type.

- **Comprehensive data sublanguage:** one language (like SQL) supports definition, manipulation, constraints and transactions.
- **Physical and logical data independence**, as described in Chapter 1.

Summary

- The relational model stores data as relations (tables) of tuples (rows) and attributes (columns).
- Relations have no duplicate tuples, unordered rows and atomic values.
- A super key uniquely identifies tuples; a candidate key is a minimal super key; the primary key is the chosen candidate key.
- Foreign keys link tables and must satisfy referential integrity.
- Constraints include domain, key, entity integrity and referential integrity.
- Actions such as CASCADE and SET NULL control what happens when referenced rows change.

Practice Questions

1. Define degree and cardinality. What are the degree and cardinality of a table with 5 columns and 200 rows?

2. In EMPLOYEE(emp_id, email, name, phone), identify the candidate keys, a sensible primary key and the alternate keys.
3. Why must a primary key never be NULL?
4. Explain referential integrity and the effect of ON DELETE CASCADE with an example.
5. Is {roll_no, name} a candidate key if roll_no alone is unique? Why or why not?

Relational Algebra

SQL is what you type, but relational algebra is what the database thinks. Relational algebra is a small set of operations that take one or two relations as input and produce a new relation as output. Every SQL query is translated into relational algebra inside the DBMS before it is optimised and run. Learning it will make SQL much easier to understand.

Why Learn Relational Algebra?

- It is **procedural**: an expression describes the steps needed to get a result.
- It is **closed**: every operation returns a relation, so operations can be chained like arithmetic.
- It is the **foundation of query optimisation**. The DBMS rewrites algebra expressions into cheaper equivalent ones.

We will use two example relations throughout this chapter.

STUDENT

roll	name	branch	cgpa
101	Riya	CSE	8.9
102	Arjun	IT	7.4
103	Meera	CSE	9.3
104	Kabir	ECE	6.8

ENROLLS

roll	course
101	DBMS
101	OS
103	DBMS
104	DSA

Unary Operations

Selection (sigma)

Selection picks the *rows* that satisfy a condition. It is written as sigma with the condition as a subscript.

```
sigma[branch = 'CSE'] (STUDENT)
```

Result: the rows for Riya and Meera. Conditions can use comparison operators (=, <>, <, >, <=, >=) and can be combined with AND, OR and NOT:

```
sigma[branch = 'CSE' AND cgpa > 9] (STUDENT)    -- only  
Meera
```

Selection never changes the columns; it only filters rows. The number of rows in the result is at most the number in the input.

Projection (π)

Projection picks the *columns* you want and discards the rest.

```
 $\pi$ [name, cgpa] (STUDENT)
```

Because a relation is a set, projection **removes duplicate rows**. Projecting STUDENT on branch gives only three rows: CSE, IT and ECE, even though CSE appears twice in the original.

Key idea: Selection works horizontally (rows). Projection works vertically (columns). In SQL, selection is the WHERE clause and projection is the SELECT list.

Combining Selection and Projection

Operations can be nested. To find the names of CSE students:

```
 $\pi$ [name] (  $\sigma$ [branch = 'CSE'] (STUDENT) )
```

Read it from the inside out: first keep the CSE rows, then keep only the name column.

Rename (ρ)

Rename gives a relation or its attributes a new name. It is needed when a relation is combined with itself, for example to compare two students.

```
 $\rho$ [S2] (STUDENT)
```

Set Operations

Set operations combine two relations that are

union-compatible: they must have the same number of attributes, with matching domains.

- **Union ($R \cup S$):** all tuples in R or S or both, with duplicates removed.
- **Intersection ($R \cap S$):** tuples in both R and S.
- **Difference ($R - S$):** tuples in R but not in S.

Example: roll numbers of students enrolled in DBMS *or* OS:

```
pi[roll](sigma[course='DBMS'](ENROLLS))  $\cup$   
pi[roll](sigma[course='OS'](ENROLLS))
```

Result: 101, 103. Roll numbers enrolled in DBMS but *not* OS: only 103.

Union and intersection are commutative ($R \cup S = S \cup R$), but difference is not: $R - S$ is usually different from $S - R$.

Cartesian Product

The **Cartesian product** ($R \times S$) pairs every tuple of R with every tuple of S. If R has 4 rows and S has 4 rows, the product has 16 rows, with all the columns of both.

On its own, the product is rarely useful, because most of the pairs are meaningless: it pairs Riya with Kabir's enrolment. Its real value is as a building block for joins.

Joins

A **join** combines related tuples from two relations. It is equivalent to a Cartesian product followed by a selection, but it is so common that it has its own operator.

Theta Join

A **theta join** combines tuples that satisfy any condition theta:

```
STUDENT ⋈[STUDENT.roll = ENROLLS.roll] ENROLLS
```

Equijoin

An **equijoin** is a theta join that only uses equality. The example above is an equijoin. The result contains both roll columns, which are identical.

Natural Join

A **natural join** automatically joins on all attributes with the same name and keeps only one copy of each. `STUDENT ⋈ ENROLLS` joins on roll:

roll	name	branch	cgpa	course
101	Riya	CSE	8.9	DBMS
101	Riya	CSE	8.9	OS
103	Meera	CSE	9.3	DBMS
104	Kabir	ECE	6.8	DSA

Arjun (102) disappears because he has no enrolments. Rows without a match are dropped in an ordinary (inner) join.

Outer Joins

Outer joins keep unmatched rows and fill missing values with NULL.

- **Left outer join:** keeps all rows of the left relation. Arjun would appear with course = NULL.
- **Right outer join:** keeps all rows of the right relation.
- **Full outer join:** keeps unmatched rows from both sides.

Key idea: Use an inner join when you only want matches. Use a left outer join when you want "everyone, plus their matches if any", such as "all students and their courses, including students with no courses".

Division

Division answers "for all" questions. Suppose we want students who are enrolled in *every* course listed in a relation COURSES(course) containing DBMS and OS.

ENROLLS $\div$ COURSES

The result contains roll numbers that appear in ENROLLS paired with *every* course in COURSES. Only 101 (Riya) qualifies. Division is less common than the other operators, but whenever you see "all" or "every" in a query, think of it.

Writing Queries in Relational Algebra

Let us translate a few English questions.

1. *Names of students with CGPA above 8.* $\pi[\text{name}] (\sigma[\text{cgpa} > 8] (\text{STUDENT}))$
2. *Names of students enrolled in DBMS.* $\pi[\text{name}] (\sigma[\text{course} = \text{'DBMS'}] (\text{STUDENT} \bowtie \text{ENROLLS}))$
3. *Roll numbers of students not enrolled in any course.*
 $\pi[\text{roll}](\text{STUDENT}) - \pi[\text{roll}](\text{ENROLLS})$ gives 102.

Query Trees

A relational algebra expression can be drawn as a **query tree**: leaves are relations and inner nodes are operators. The DBMS optimiser rearranges this tree to make it cheaper. A classic rule is "**push selections down**": filter rows as early as possible, so later joins process fewer rows. In query 2, selecting DBMS rows from ENROLLS *before* the join is far cheaper than joining everything first.

Relational Calculus in Brief

Relational algebra says *how* to compute a result. **Relational calculus** says *what* result you want, using logic, without giving steps. It comes in two forms:

- **Tuple relational calculus:** $\{ t \mid t \in \text{STUDENT AND } t.\text{cgpa} > 8 \}$ means "all tuples t in STUDENT whose cgpa is above 8".

- **Domain relational calculus:** variables range over attribute values instead of whole tuples.

SQL is closer to calculus in spirit, because it is *declarative*: you describe the result and the DBMS decides the steps. Codd proved that relational algebra and safe relational calculus have the same expressive power.

Summary

- Relational algebra is a procedural, closed set of operations on relations.
- Selection filters rows; projection keeps columns and removes duplicates; rename changes names.
- Union, intersection and difference need union-compatible relations.
- The Cartesian product pairs all tuples; joins combine only related tuples.
- Natural join matches on common attribute names; outer joins keep unmatched rows with NULLs.
- Division answers "for all" queries.
- Optimisers rewrite query trees, for example by pushing selections down.

Practice Questions

1. What is the difference between selection and projection?
Give their SQL equivalents.
2. Why does projection remove duplicates? Show an example.
3. If R has 5 rows and S has 3 rows, how many rows does $R \times S$ have?
4. Write relational algebra for: names of CSE students enrolled in OS.
5. When would you use a left outer join instead of a natural join?

SQL Basics

SQL (pronounced "sequel" or "S-Q-L") is the standard language for relational databases. It was developed at IBM in the 1970s and is now supported by every major DBMS. The good news is that the core of SQL is small and readable. This chapter teaches you to create tables, add and change data, and write simple queries.

Data Types

Every column has a data type that limits what it can store. Exact names vary slightly between systems, but the common ones are:

Type	Stores	Example
INT / INTEGER	Whole numbers	42
DECIMAL(p, s)	Exact decimals with p digits, s after the point	8.75
FLOAT / REAL	Approximate decimals	3.14159
CHAR(n)	Fixed-length text, padded with spaces	'CSE '
VARCHAR(n)	Variable-length text up to n characters	'Riya'
DATE	A calendar date	'2026-09-26'
TIMESTAMP	Date and time	'2026-09-26 10:30:00'
BOOLEAN	True or false	TRUE

Key idea: Use DECIMAL, not FLOAT, for money. Floating-point numbers cannot store values like 0.1 exactly, which causes rounding errors in financial totals.

Creating Tables (DDL)

```
CREATE TABLE course (  
    course_id VARCHAR(8) PRIMARY KEY,  
    title     VARCHAR(60) NOT NULL,  
    credits   INT          CHECK (credits BETWEEN 1 AND 6),  
    dept_id   VARCHAR(5) REFERENCES department(dept_id)  
);
```

The common column constraints are:

- PRIMARY KEY: unique and not NULL.
- NOT NULL: a value is required.
- UNIQUE: no two rows may share the value.
- CHECK (condition): the value must satisfy a condition.
- DEFAULT value: used when no value is given.
- REFERENCES table(column): a foreign key.

Changing and Removing Tables

```
ALTER TABLE course ADD COLUMN semester INT;
ALTER TABLE course DROP COLUMN semester;
ALTER TABLE course RENAME TO subject;

DROP TABLE subject;           -- removes the table and all its
                                data
TRUNCATE TABLE enrolls;       -- removes all rows but keeps the
                                table
```

DROP removes the structure itself. TRUNCATE empties the table quickly. DELETE (below) removes chosen rows and can be rolled back inside a transaction.

Adding, Changing and Removing Data (DML)

INSERT

```
INSERT INTO student (roll_no, name, branch, cgpa)
VALUES (101, 'Riya', 'CSE', 8.9);

-- several rows at once
INSERT INTO student (roll_no, name, branch, cgpa) VALUES
    (102, 'Arjun', 'IT', 7.4),
    (103, 'Meera', 'CSE', 9.3);
```

Always list the column names. Your insert will then keep working even if someone later adds a new column.

UPDATE

```
UPDATE student
SET cgpa = 9.0
WHERE roll_no = 101;
```

Key idea: An UPDATE or DELETE without a WHERE clause affects **every row** in the table. Always write the WHERE clause first, and run the same condition as a SELECT to check which rows it matches.

DELETE

```
DELETE FROM student WHERE branch = 'ECE';
```

Querying Data with SELECT

The SELECT statement is the heart of SQL. Its basic shape is:

```
SELECT columns  
FROM table  
WHERE condition  
ORDER BY column;
```

Selecting Columns

```
SELECT * FROM student;           -- every column  
SELECT name, cgpa FROM student;  -- chosen columns  
SELECT name, cgpa * 9.5 AS percentage FROM student;
```

AS gives a column an **alias**, a friendlier name in the result.

Removing Duplicates

```
SELECT DISTINCT branch FROM student;
```

Unlike relational algebra, SQL keeps duplicate rows unless you ask for DISTINCT.

Filtering with WHERE

```
SELECT name FROM student WHERE cgpa >= 8.5;
SELECT name FROM student WHERE branch = 'CSE' AND cgpa > 9;
SELECT name FROM student WHERE branch IN ('CSE', 'IT');
SELECT name FROM student WHERE cgpa BETWEEN 7 AND 8.5;    --
    inclusive
```

Pattern Matching with LIKE

% matches any sequence of characters; _ matches exactly one character.

```
SELECT name FROM student WHERE name LIKE 'M%';    -- starts
    with M
SELECT name FROM student WHERE name LIKE '%ya';    -- ends
    with ya
SELECT name FROM student WHERE name LIKE '_r%';    -- second
    letter is r
```

Working with NULL

NULL means "unknown", so `cgpa = NULL` is never true, not even for rows where `cgpa` is NULL. Use `IS NULL` and `IS NOT NULL` instead.

```
SELECT name FROM student WHERE phone IS NULL;
```

Any arithmetic with NULL gives NULL: `5 + NULL` is NULL. The function `COALESCE(phone, 'not given')` replaces NULL with a default value.

Sorting with ORDER BY

```
SELECT name, cgpa FROM student ORDER BY cgpa DESC;  
SELECT name, branch, cgpa FROM student ORDER BY branch ASC,  
cgpa DESC;
```

ASC (ascending) is the default. With two sort keys, rows are sorted by branch first, and by cgpa within each branch.

Limiting Results

```
SELECT name, cgpa FROM student ORDER BY cgpa DESC LIMIT 3;  
-- top three
```

Some systems use `FETCH FIRST 3 ROWS ONLY` or `TOP 3` instead of `LIMIT`.

Built-in Functions

Function	Purpose	Example result
UPPER(name)	Uppercase text	'RIYA'
LENGTH(name)	Number of characters	4
SUBSTRING(name, 1, 2)	Part of a string	'Ri'
ROUND(cgpa, 1)	Round a number	8.9
CURRENT_DATE	Today's date	2026-09-26
COALESCE(a, b)	First non-NULL value	b if a is NULL

How SQL Maps to Relational Algebra

```
SELECT DISTINCT name  
FROM student  
WHERE branch = 'CSE';
```

This is exactly $\pi[\text{name}] (\sigma[\text{branch} = \text{'CSE'}] (\text{STUDENT}))$. The FROM clause names the relation, WHERE is selection and the SELECT list is projection. DISTINCT is needed to get the set behaviour of algebra.

The Logical Order of a Query

We write SELECT first, but the DBMS logically processes the clauses in this order:

1. FROM: find the table(s)
2. WHERE: filter rows
3. SELECT: choose and compute columns
4. DISTINCT: remove duplicates
5. ORDER BY: sort
6. LIMIT: cut the result

This explains a common error: you usually cannot use a column alias from SELECT inside WHERE, because WHERE runs first.

Summary

- Choose data types carefully; use DECIMAL for money.

- DDL (CREATE, ALTER, DROP, TRUNCATE) defines structure; DML (INSERT, UPDATE, DELETE, SELECT) works with data.
- Always use a WHERE clause with UPDATE and DELETE unless you truly mean every row.
- WHERE supports comparisons, AND/OR, IN, BETWEEN, LIKE and IS NULL.
- ORDER BY sorts, DISTINCT removes duplicates and LIMIT restricts the number of rows.
- Logically, FROM and WHERE are processed before SELECT.

Practice Questions

1. Write a CREATE TABLE statement for BOOK(isbn, title, author, price, published_on) with sensible types and constraints.
2. What is the difference between DROP, TRUNCATE and DELETE?
3. Why does WHERE phone = NULL return no rows? What should you write instead?
4. Write a query to list the names of the five students with the lowest CGPA in the IT branch.
5. Write queries to find students whose names contain the letter "a" as the second-last character.

Advanced SQL: Joins, Grouping and Subqueries

Real questions rarely involve a single table. "Which department has the highest average CGPA?" or "Which students have not submitted any assignment?" need data from several tables, summarised and filtered. This chapter covers the SQL features that make such questions easy: joins, aggregate functions, grouping, subqueries and views.

We will use these tables:

```
student(roll_no, name, dept_id, cgpa)
department(dept_id, dept_name)
course(course_id, title, credits)
enrolls(roll_no, course_id, grade)
```

Joins in SQL

Inner Join

An inner join returns only rows that have a match in both tables.

```
SELECT s.name, d.dept_name
FROM student s
JOIN department d ON s.dept_id = d.dept_id;
```

`s` and `d` are **table aliases**, which keep queries short and are required when a column name exists in both tables. `JOIN` on its own means `INNER JOIN`.

Joining Three Tables

```
SELECT s.name, c.title, e.grade
FROM student s
JOIN enrolls e ON e.roll_no = s.roll_no
JOIN course c ON c.course_id = e.course_id
WHERE c.title = 'DBMS';
```

Joins are applied one after another. Each `ON` clause links the new table to the ones already joined.

Outer Joins

```
-- every student, with their courses if they have any
SELECT s.name, e.course_id
FROM student s
LEFT JOIN enrolls e ON e.roll_no = s.roll_no;
```

Students with no enrolments appear once, with `course_id = NULL`. A `RIGHT JOIN` keeps all rows of the right table, and a `FULL OUTER JOIN` keeps unmatched rows from both.

Key idea: To find rows *without* a match, use a `LEFT JOIN` and then keep the rows where the right side is `NULL`. This "anti-join" pattern is very common.

```
-- students not enrolled in anything
SELECT s.name
FROM student s
LEFT JOIN enrolls e ON e.roll_no = s.roll_no
WHERE e.roll_no IS NULL;
```

Self Join

A table can be joined with itself. Suppose `employee(emp_id, name, manager_id)`:

```
SELECT e.name AS employee, m.name AS manager
FROM employee e
LEFT JOIN employee m ON e.manager_id = m.emp_id;
```

Cross Join

`CROSS JOIN` produces the Cartesian product: every row paired with every row. It is occasionally useful, for example to create every combination of sizes and colours for a product catalogue.

Aggregate Functions

Aggregate functions summarise many rows into one value.

Function	Returns
<code>COUNT(*)</code>	Number of rows
<code>COUNT(column)</code>	Number of non-NULL values in the column
<code>SUM(column)</code>	Total
<code>AVG(column)</code>	Average (NULLs are ignored)
<code>MIN(column) / MAX(column)</code>	Smallest / largest value

```
SELECT COUNT(*) AS students, AVG(cgpa) AS avg_cgpa, MAX(cgpa)
      AS topper
FROM student;
```

Note that `COUNT(*)` counts rows, while `COUNT(phone)` counts only rows where `phone` is not `NULL`. `COUNT(DISTINCT dept_id)` counts different departments.

GROUP BY

`GROUP BY` splits rows into groups and computes aggregates for each group.

```
SELECT dept_id, COUNT(*) AS students, ROUND(AVG(cgpa), 2) AS  
       avg_cgpa  
FROM student  
GROUP BY dept_id;
```

Result, one row per department:

dept_id	students	avg_cgpa
CSE	42	8.12
IT	38	7.86
ECE	35	7.54

Key idea: In a grouped query, every column in the `SELECT` list must either appear in `GROUP BY` or be inside an aggregate function. Otherwise SQL cannot know which row's value to show.

HAVING

`WHERE` filters rows *before* grouping. `HAVING` filters groups *after* grouping.

```
SELECT dept_id, AVG(cgpa) AS avg_cgpa
FROM student
WHERE cgpa IS NOT NULL
GROUP BY dept_id
HAVING AVG(cgpa) > 8;
```

This finds departments whose average CGPA is above 8. You cannot write `WHERE AVG(cgpa) > 8`, because averages do not exist until the groups are formed.

The full logical order of a query is now:

1. FROM and JOINS
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. DISTINCT
7. ORDER BY
8. LIMIT

Subqueries

A **subquery** is a query inside another query.

Subquery Returning a Single Value

```
-- students above the overall average
SELECT name, cgpa
FROM student
WHERE cgpa > (SELECT AVG(cgpa) FROM student);
```

Subquery with IN

```
-- students enrolled in DBMS
SELECT name
FROM student
WHERE roll_no IN (SELECT roll_no FROM enrolls WHERE course_id
    = 'CS301');
```

EXISTS and NOT EXISTS

EXISTS is true if the subquery returns at least one row.

```
-- students who have at least one grade 'A'
SELECT s.name
FROM student s
WHERE EXISTS (
    SELECT 1 FROM enrolls e
    WHERE e.roll_no = s.roll_no AND e.grade = 'A'
);
```

This is a **correlated subquery**: it refers to *s* from the outer query, so it is conceptually re-run for each student. `NOT EXISTS` is the safest way to express "has no matching row".

Key idea: Be careful with `NOT IN` when the subquery can return `NULL`. If any value is `NULL`, `NOT IN` returns no rows at all. `NOT EXISTS` does not have this problem.

ANY and ALL

```
SELECT name FROM student
WHERE cgpa > ALL (SELECT cgpa FROM student WHERE dept_id =
    'IT');
```

This finds students who beat every IT student. > ANY would mean "beats at least one".

Set Operations

```
SELECT roll_no FROM enrolls WHERE course_id = 'CS301'
UNION
SELECT roll_no FROM enrolls WHERE course_id = 'CS302';
```

UNION removes duplicates; UNION ALL keeps them and is faster. INTERSECT and EXCEPT (called MINUS in Oracle) also exist.

Views

A **view** is a saved query that behaves like a virtual table.

```
CREATE VIEW cse_toppers AS
SELECT roll_no, name, cgpa
FROM student
WHERE dept_id = 'CSE' AND cgpa >= 9;

SELECT * FROM cse_toppers ORDER BY cgpa DESC;
```

Views are useful to:

- **simplify** complicated queries by giving them a name,

- **secure** data by showing users only certain rows or columns, and
- **provide logical data independence**, since programs can keep using the view even if the base tables change.

Most views store no data of their own; the query runs whenever the view is used. A **materialized view** stores the result physically and must be refreshed, which trades freshness for speed.

Common Table Expressions

A **CTE** (the `WITH` clause) names a temporary result for use in one query. It makes long queries much easier to read.

```
WITH dept_avg AS (  
    SELECT dept_id, AVG(cgpa) AS avg_cgpa  
    FROM student  
    GROUP BY dept_id  
)  
SELECT s.name, s.dept_id, s.cgpa  
FROM student s  
JOIN dept_avg d ON d.dept_id = s.dept_id  
WHERE s.cgpa > d.avg_cgpa;
```

This lists students who are above their own department's average.

Window Functions in Brief

Window functions compute values across related rows *without* collapsing them into groups.

```
SELECT name, dept_id, cgpa,  
       RANK() OVER (PARTITION BY dept_id ORDER BY cgpa DESC)  
       AS dept_rank  
FROM student;
```

Every student keeps their own row, and gains their rank within their department. `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()` and running totals such as `SUM(x) OVER (ORDER BY date)` are common window functions.

Summary

- Inner joins keep matches; outer joins also keep unmatched rows; a `LEFT JOIN` with an `IS NULL` check finds missing matches.
- Aggregates summarise rows; `GROUP BY` computes them per group; `HAVING` filters groups.
- Subqueries can return a value, a list (`IN`) or a yes/no answer (`EXISTS`). Prefer `NOT EXISTS` to `NOT IN`.
- `UNION`, `INTERSECT` and `EXCEPT` combine query results.
- Views save queries, simplify access and improve security; CTEs make long queries readable.
- Window functions rank and total rows without grouping them away.

Practice Questions

1. Write a query listing each course title with the number of students enrolled, including courses with zero students.
2. What is the difference between WHERE and HAVING? Give an example that needs both.
3. Find the name of the student(s) with the highest CGPA in each department.
4. Explain why `NOT IN` can return an empty result when the subquery contains NULL.
5. Create a view that shows only name and dept_name for all students, hiding CGPA.

Functional Dependencies and Normalization

A badly designed table causes trouble even if every query is written perfectly. Data gets repeated, updates go wrong, and some facts cannot be stored at all. **Normalization** is a step-by-step method for splitting tables so that each fact is stored exactly once. It is built on the idea of functional dependencies.

A Badly Designed Table

Consider this single table that a college might start with:

```
STUDENT_COURSE(roll_no, name, dept, hod, course_id,
               course_title, grade)
```

101	Riya	CSE	Dr. Rao	CS301	DBMS	A
101	Riya	CSE	Dr. Rao	CS302	OS	B
103	Meera	CSE	Dr. Rao	CS301	DBMS	A
104	Kabir	ECE	Dr. Iyer	EC201	DSP	B

It looks harmless, but it has three kinds of **anomalies**.

- **Update anomaly.** The head of CSE is stored in many rows. If Dr. Rao is replaced, we must change every CSE row. Missing one leaves the data inconsistent.

- **Insertion anomaly.** We cannot record a new course until some student enrolls in it, because roll_no is part of the key and cannot be NULL.
- **Deletion anomaly.** If Kabir leaves and we delete his row, we also lose the fact that ECE's head is Dr. Iyer and that the course DSP exists.

Key idea: Anomalies happen when one table stores facts about several different things: students, departments, courses and enrolments. Normalization separates those things.

Functional Dependencies

A **functional dependency (FD)**, written $X \rightarrow Y$, means: *if two rows agree on X, they must agree on Y*. In other words, X determines Y.

In our table:

- roll_no $\rightarrow$ name, dept (a roll number determines the student's name and department)
- dept $\rightarrow$ hod (a department has one head)
- course_id $\rightarrow$ course_title
- {roll_no, course_id} $\rightarrow$ grade (a grade belongs to a student in a course)

X is called the **determinant**. FDs come from the meaning of the data, not from looking at a few rows. Just because every current student has a unique name does not mean $\text{name} \rightarrow \text{roll_no}$ is a real rule.

Types of Dependencies

- **Trivial FD:** Y is part of X, such as $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$. It is always true and tells us nothing.
- **Full functional dependency:** Y depends on the *whole* of X, not just part of it. $\{\text{roll_no}, \text{course_id}\} \rightarrow \text{grade}$ is full.
- **Partial dependency:** Y depends on only part of a composite key. $\{\text{roll_no}, \text{course_id}\} \rightarrow \text{name}$ is partial, because roll_no alone determines name.
- **Transitive dependency:** $X \rightarrow Y$ and $Y \rightarrow Z$, so $X \rightarrow Z$ indirectly. $\text{roll_no} \rightarrow \text{dept}$ and $\text{dept} \rightarrow \text{hod}$ give $\text{roll_no} \rightarrow \text{hod}$ transitively.

Armstrong's Axioms

These rules let us derive new FDs from known ones:

1. **Reflexivity:** if Y is a subset of X, then $X \rightarrow Y$.
2. **Augmentation:** if $X \rightarrow Y$, then $XZ \rightarrow YZ$.
3. **Transitivity:** if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Useful derived rules include **union** ($X \rightarrow Y$ and $X \rightarrow Z$ give $X \rightarrow YZ$) and **decomposition** ($X \rightarrow YZ$ gives $X \rightarrow Y$ and $X \rightarrow Z$).

Attribute Closure

The **closure** of X, written X^+ , is the set of all attributes that X determines. To compute it, start with X and keep adding the right side of any FD whose left side is already included.

Example with the FDs above, computing $\{\text{roll_no}\}^+$:

1. Start: $\{\text{roll_no}\}$
2. $\text{roll_no} \rightarrow \text{name, dept}$ adds: $\{\text{roll_no, name, dept}\}$
3. $\text{dept} \rightarrow \text{hod}$ adds: $\{\text{roll_no, name, dept, hod}\}$
4. No more FDs apply.

Since $\{\text{roll_no}\}^+$ does not contain `course_id` or `grade`, `roll_no` alone is not a key. But $\{\text{roll_no, course_id}\}^+$ contains every attribute, so it is a **candidate key**. Closures are the standard way to test whether a set of attributes is a key.

The Normal Forms

Each normal form removes a particular kind of problem. A table in a higher normal form is automatically in all lower ones.

First Normal Form (1NF)

A relation is in **1NF** if every attribute holds only **atomic** (single) values, with no repeating groups or lists.

Not in 1NF:

roll_no		name		phones
101		Riya		98200 11111, 98200 22222

Fix: store one phone per row, usually in a separate table
STUDENT_PHONE(roll_no, phone).

Second Normal Form (2NF)

A relation is in **2NF** if it is in 1NF and has **no partial dependencies**: every non-key attribute depends on the *whole* of every candidate key.

Our table has key {roll_no, course_id}, but name and dept depend only on roll_no, and course_title depends only on course_id. We split it:

```
STUDENT(roll_no, name, dept, hod)
COURSE(course_id, course_title)
ENROLLS(roll_no, course_id, grade)
```

2NF only matters when a key is composite. A table with a single-attribute key is automatically in 2NF.

Third Normal Form (3NF)

A relation is in **3NF** if it is in 2NF and has **no transitive dependencies**: non-key attributes depend only on the key, not on other non-key attributes.

STUDENT still has roll_no -> dept -> hod. We split again:

```
STUDENT(roll_no, name, dept)
DEPARTMENT(dept, hod)
```

A handy way to remember 3NF: every non-key attribute must depend on "the key, the whole key, and nothing but the key".

Formally, a relation is in 3NF if for every non-trivial FD $X \rightarrow A$, either X is a super key or A is part of some candidate key.

Boyce-Codd Normal Form (BCNF)

BCNF is a slightly stricter version of 3NF: for every non-trivial FD $X \rightarrow Y$, **X must be a super key**. There is no exception for key attributes.

Example: TEACHES(student, subject, teacher) where each teacher teaches one subject (teacher $\rightarrow$ subject), and each student has one teacher per subject ({student, subject} $\rightarrow$ teacher). The table is in 3NF, because subject is part of a candidate key, but not in BCNF, because teacher is not a super key. Splitting into (teacher, subject) and (student, teacher) achieves BCNF.

Key idea: 3NF can always be reached without losing any functional dependency. BCNF removes more redundancy but occasionally makes an FD impossible to check within a single table. In practice, most designers aim for 3NF or BCNF.

Higher Normal Forms

4NF deals with multivalued dependencies: for example, a table storing a teacher's independent sets of subjects and hobbies together. **5NF** deals with join dependencies. They are rarely needed in everyday design.

Good Decompositions

When we split a table, the split must satisfy two properties.

1. **Lossless join.** Joining the pieces must give back exactly the original table, with no extra, fake rows. A split of R into R_1 and R_2 is lossless if the common attributes form a key of R_1 or of R_2 . Splitting `STUDENT` into `STUDENT(roll_no, name, dept)` and `DEPARTMENT(dept, hod)` is lossless, because `dept` is the key of `DEPARTMENT`.
2. **Dependency preservation.** Every original FD can still be checked within a single piece, without joining.

Lossless join is essential; dependency preservation is highly desirable.

Denormalization

Normalization reduces redundancy but increases the number of joins. In read-heavy systems, such as reporting dashboards, designers sometimes **denormalize** on purpose, keeping a copy of the department name in the student table for example, to make queries faster. This is a conscious trade-off, not a design mistake, and it must be managed carefully to avoid anomalies.

Summary

- Poor design causes update, insertion and deletion anomalies.
- A functional dependency $X \rightarrow Y$ means X determines Y; closures find keys.
- 1NF: atomic values. 2NF: no partial dependencies. 3NF: no transitive dependencies. BCNF: every determinant is a super key.
- Decompositions must be lossless and should preserve dependencies.
- Denormalization deliberately adds redundancy for read performance.

Practice Questions

1. Explain the three types of anomalies using an example table of your own.

2. Given $R(A, B, C, D)$ with $A \rightarrow B$ and $B \rightarrow C$, compute A^+ and find the candidate key if D is determined by A .
3. What is the difference between a partial and a transitive dependency?
4. Normalize $ORDER(order_id, customer_id, customer_name, product_id, product_name, qty)$ to 3NF.
5. Why is 3NF sometimes preferred over BCNF?

Transactions and the ACID Properties

Imagine you transfer 5,000 rupees to a friend. Your bank must subtract the money from your account and add it to your friend's account. If the system crashes after the first step, the money vanishes. If two transfers from your account run at the same moment, you might overspend. **Transactions** are how a DBMS prevents these disasters.

What Is a Transaction?

A **transaction** is a sequence of database operations that forms one logical unit of work. It must either happen **completely** or **not at all**.

```
BEGIN;  
UPDATE account SET balance = balance - 5000 WHERE acc_no =  
    'A101';  
UPDATE account SET balance = balance + 5000 WHERE acc_no =  
    'B202';  
COMMIT;
```

BEGIN starts the transaction. COMMIT makes all its changes permanent. If anything goes wrong, ROLLBACK undoes every change made since BEGIN.

Read and Write Operations

Inside the DBMS, a transaction is described by its reads and writes:

- `read(X)` copies data item X from the database into the transaction's workspace.
- `write(X)` copies the new value of X back to the database.

The transfer above is: `read(A)`, $A = A - 5000$, `write(A)`, `read(B)`, $B = B + 5000$, `write(B)`.

The ACID Properties

Every transaction must satisfy four properties, remembered as **ACID**.

Atomicity

All or nothing. Either every operation of the transaction takes effect, or none does. If the system fails after `write(A)` but before `write(B)`, the DBMS must undo `write(A)` when it recovers.

Atomicity is handled by the **recovery system**, usually through a log (Chapter 10).

Consistency

A transaction takes the database from one valid state to another. In a transfer, the total money in both accounts must be the same before and after. Consistency depends partly on the programmer writing correct transactions and partly on the DBMS enforcing constraints.

Isolation

Concurrent transactions must not interfere with each other. The result of running transactions at the same time must be the same as running them one after another in *some* order. If another transaction reads the accounts halfway through a transfer, it must not see money that has left A but not yet reached B. Isolation is handled by **concurrency control** (Chapter 9).

Durability

Once committed, changes survive failures. After the bank says "transfer successful", a power cut must not undo it. Durability is achieved by writing changes (or log records describing them) to non-volatile storage before confirming the commit.

Key idea: Atomicity and durability are about *failures*. Isolation is about *other users*. Consistency is about *correctness* of the data.

Property	Question it answers	Mainly ensured by
Atomicity	Did all of it happen, or none?	Recovery manager (undo)
Consistency	Are the rules still true?	Programmer + constraints
Isolation	Did others see half-finished work?	Concurrency control
Durability	Will it survive a crash?	Recovery manager (redo)

Transaction States

A transaction moves through these states:

1. **Active:** it is executing its operations.
2. **Partially committed:** its last operation has run, but changes may still be only in memory.
3. **Committed:** its changes are safely recorded and permanent.
4. **Failed:** an error means it cannot continue normally.
5. **Aborted:** its changes have been rolled back. The system may then restart the transaction or kill it.

A transaction that has committed or aborted is said to be **terminated**.

Why Run Transactions Concurrently?

It would be simplest to run one transaction at a time, but real systems run many at once because:

- **Throughput improves.** While one transaction waits for disk, another can use the CPU.
- **Waiting time drops.** A short transaction does not have to wait behind a long report.

Concurrency brings risks, however. Without control, these problems appear.

Lost Update

T1 and T2 both read a seat count of 10. T1 books one seat and writes 9. T2 books one seat and also writes 9. Two seats were sold, but the count only dropped by one. T1's update is **lost**.

Dirty Read

T1 changes a price and T2 reads the new price. Then T1 fails and rolls back. T2 has used a value that never officially existed. This is a **dirty read** (reading uncommitted data).

Unrepeatable Read

T1 reads a balance, T2 updates and commits it, and T1 reads the balance again and gets a different value within the same transaction.

Phantom Read

T1 counts students with CGPA above 9 and gets 12. T2 inserts a new such student and commits. T1 runs the same query again and now gets 13. A **phantom** row appeared.

Schedules

A **schedule** is the order in which the operations of concurrent transactions are executed.

- A **serial schedule** runs transactions one after another with no interleaving. It is always correct but slow.
- A **concurrent (non-serial) schedule** interleaves operations.

Serializability

A concurrent schedule is **serializable** if its effect is the same as some serial schedule. Serializability is the formal definition of "correct" isolation.

Conflict serializability is the version most often tested. Two operations **conflict** if they belong to different transactions, access the same data item, and at least one of them is a write. The conflicting pairs are read-write, write-read and write-write.

To test a schedule, build a **precedence graph**:

1. Draw a node for each transaction.
2. For each conflicting pair where T_i 's operation comes before T_j 's, draw an edge $T_i \rightarrow T_j$.
3. If the graph has **no cycle**, the schedule is conflict serializable. A topological order of the graph gives an equivalent serial schedule.

```
Schedule S:  r1(A)  w1(A)  r2(A)  w2(A)  r1(B)  w1(B)
```

```
Conflicts: w1(A) before r2(A)  ->  edge T1 -> T2
```

```
           w1(A) before w2(A)  ->  edge T1 -> T2
```

```
No edge from T2 to T1, so no cycle: S is equivalent to T1  
then T2.
```

Recoverable Schedules

A schedule is **recoverable** if a transaction commits only after every transaction whose data it read has committed. If T2 reads T1's write and commits first, and then T1 aborts, T2 has committed a dirty read that cannot be undone.

A **cascading rollback** happens when one abort forces several other transactions to abort, because they read its uncommitted data. **Cascadeless** schedules avoid this by allowing reads only of committed data.

Isolation Levels in SQL

Full serializability can be slow, so SQL lets you choose weaker levels:

Isolation level	Dirty read	Unrepeatable read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Many databases default to READ COMMITTED. Banking and booking systems often need SERIALIZABLE for critical operations.

Summary

- A transaction is a unit of work that must happen completely or not at all.
- ACID: atomicity, consistency, isolation, durability.
- States: active, partially committed, committed, failed, aborted.
- Uncontrolled concurrency causes lost updates, dirty reads, unrepeatable reads and phantoms.
- A schedule is correct if it is serializable; use a precedence graph to test conflict serializability.
- Recoverable and cascadeless schedules limit the damage when a transaction aborts.
- SQL isolation levels trade correctness for speed.

Practice Questions

1. Explain each ACID property using a railway ticket booking example.
2. Describe the lost update problem with a timeline of two transactions.
3. Test whether $r_1(X) \ r_2(X) \ w_1(X) \ w_2(X)$ is conflict serializable using a precedence graph.

4. What is a cascading rollback, and how do cascadeless schedules prevent it?
5. Which isolation level would you choose for a report that counts students, and why?

Concurrency Control

In the previous chapter we saw what can go wrong when transactions run at the same time, and we defined serializability as the goal. This chapter explains *how* a DBMS actually achieves it. The main techniques are locking, timestamps and multiversioning. We will also study deadlocks, which appear as a side effect of locking.

Lock-Based Protocols

A **lock** is a marker that a transaction places on a data item to control access by others.

Types of Locks

- **Shared lock (S):** allows the holder to *read* the item. Many transactions can hold shared locks on the same item at once.
- **Exclusive lock (X):** allows the holder to *read and write* the item. Only one transaction can hold it, and no one else can hold any lock on that item at the same time.

The **compatibility matrix** summarises which requests can be granted:

Requested \ Held	Shared	Exclusive
Shared	Yes	No
Exclusive	No	No

If a lock cannot be granted, the requesting transaction **waits** until the other transaction releases its lock.

Key idea: Readers do not block other readers, but a writer blocks everyone and everyone blocks a writer.

Locking Alone Is Not Enough

Simply locking before each access and unlocking right after does not guarantee serializability. T1 could lock A, update it, unlock it, and only later update B. In between, T2 could read both A and B, and see A after the transfer but B before it. We need rules about *when* locks can be released.

Two-Phase Locking (2PL)

The **two-phase locking protocol** requires every transaction to work in two phases:

1. **Growing phase:** the transaction may acquire locks but may not release any.
2. **Shrinking phase:** the transaction may release locks but may not acquire any new ones.

The moment a transaction acquires its last lock is called its **lock point**. Transactions following 2PL are serializable in the order of their lock points.

```

T1: lock-X(A)  read(A)  write(A)  lock-X(B)  unlock(A)
      read(B)  write(B)  unlock(B)
      |----- growing -----|-----
      shrinking -----|

```

Problems with Basic 2PL

- **Deadlocks** can still happen (see below).
- **Cascading rollbacks** can happen: if T1 releases a lock in its shrinking phase, T2 may read T1's uncommitted data, and then T1 aborts.

Strict and Rigorous 2PL

- **Strict 2PL** holds all *exclusive* locks until the transaction commits or aborts. No one can read uncommitted writes, so cascading rollbacks disappear.
- **Rigorous 2PL** holds *all* locks, shared and exclusive, until commit or abort. Transactions are then serializable in their commit order.

Most commercial databases use strict or rigorous 2PL.

Lock Conversion

A transaction may **upgrade** a shared lock to exclusive (only in the growing phase) or **downgrade** an exclusive lock to shared (only in the shrinking phase). For example, a transaction might read a row under a shared lock, decide to change it, and then upgrade.

Deadlocks

A **deadlock** occurs when transactions wait for each other in a cycle, so none can proceed.

```
T1: lock-X(A) ... waits for lock-X(B)
T2: lock-X(B) ... waits for lock-X(A)
```

T1 holds A and wants B; T2 holds B and wants A. Both wait forever.

Deadlock Prevention

Prevention schemes make deadlock impossible. Two classic schemes use transaction timestamps, where an older transaction has a smaller timestamp.

- **Wait-die (non-preemptive):** if an *older* transaction requests a lock held by a younger one, it waits. If a *younger* transaction requests a lock held by an older one, it **dies** (aborts) and restarts later with its original timestamp.
- **Wound-wait (preemptive):** if an *older* transaction requests a lock held by a younger one, it **wounds** the younger one, forcing it to abort. If a younger one requests a lock held by an older one, it waits.

In both schemes, older transactions are favoured, so every transaction eventually becomes the oldest and finishes. Nobody starves.

Other prevention ideas include acquiring all locks at the start, or always locking items in a fixed order.

Deadlock Detection and Recovery

Instead of preventing deadlocks, the DBMS can let them happen and fix them.

1. Maintain a **wait-for graph**: draw an edge $T_i \rightarrow T_j$ when T_i is waiting for a lock held by T_j .
2. Periodically check the graph for a **cycle**. A cycle means a deadlock.
3. Choose a **victim** transaction, usually the one that has done the least work, and roll it back to break the cycle.

A simpler approach is a **timeout**: if a transaction waits longer than a set time, assume a deadlock and abort it.

Key idea: Prevention wastes some work by aborting transactions that might never have deadlocked. Detection lets transactions run freely, but must pay for checking the graph. Systems with few deadlocks usually prefer detection.

Starvation

Starvation happens when a transaction waits indefinitely, for example because it is repeatedly chosen as a deadlock victim. Fixes include counting how many times a transaction has been rolled back and never choosing it again past a limit.

Timestamp-Based Protocols

Instead of locks, each transaction receives a unique **timestamp** $TS(T)$ when it starts. The protocol ensures that conflicting operations execute in timestamp order, so the schedule is equivalent to running transactions in the order they started.

For each data item Q , the DBMS stores:

- **W-timestamp(Q):** the largest timestamp of any transaction that wrote Q .
- **R-timestamp(Q):** the largest timestamp of any transaction that read Q .

Rules for a transaction T :

1. **T reads Q:** if $TS(T) < W\text{-timestamp}(Q)$, T is trying to read a value that a younger transaction has already overwritten, so T is rolled back. Otherwise the read happens and $R\text{-timestamp}(Q)$ is updated.
2. **T writes Q:** if $TS(T) < R\text{-timestamp}(Q)$ or $TS(T) < W\text{-timestamp}(Q)$, a younger transaction has already used or replaced Q , so T is rolled back. Otherwise the write happens and $W\text{-timestamp}(Q)$ is set to $TS(T)$.

Timestamp ordering never causes deadlocks, because nobody waits. However, it can cause many rollbacks and, without extra rules, cascading aborts.

Thomas' Write Rule

A small improvement: if T tries to write Q but a younger transaction has already written Q ($TS(T) < W\text{-timestamp}(Q)$), the write is **obsolete** and can simply be ignored instead of rolling T back. This allows more schedules to succeed.

Multiversion Concurrency Control (MVCC)

MVCC keeps several versions of each data item. Writers create a new version instead of overwriting, and each reader sees the version that was current when its transaction (or statement) started.

Benefits:

- **Readers never block writers, and writers never block readers.**
- Long reports can run without holding locks that slow everyone else down.

PostgreSQL, MySQL's InnoDB engine and Oracle all use forms of MVCC. Old versions that no transaction needs any more are cleaned up by a background process (called **VACUUM** in PostgreSQL).

Optimistic Concurrency Control

Validation-based (optimistic) protocols assume conflicts are rare. Each transaction runs in three phases:

1. **Read phase:** read data and make changes in a private workspace.
2. **Validation phase:** check whether any conflicting transaction committed in the meantime.
3. **Write phase:** if validation succeeds, copy the changes into the database; otherwise restart.

This works well when most transactions only read, such as on a news website.

Lock Granularity

Locks can be placed on items of different sizes: a whole database, a table, a page or a single row. Fine-grained (row) locks allow more concurrency but cost more to manage. Coarse-grained (table) locks are cheap but block more users.

Multiple-granularity locking lets a transaction lock at the level it needs. It uses **intention locks** (IS, IX, SIX) on higher levels to announce "I hold or will hold locks further down". For example, before locking a row in exclusive mode, a transaction places an IX lock on the table.

Summary

- Shared locks allow reading by many; exclusive locks allow writing by one.

- Two-phase locking (grow then shrink) guarantees serializability; strict 2PL also prevents cascading rollbacks.
- Deadlocks can be prevented (wait-die, wound-wait) or detected with a wait-for graph and resolved by choosing a victim.
- Timestamp ordering avoids deadlocks by aborting out-of-order operations; Thomas' write rule ignores obsolete writes.
- MVCC keeps multiple versions so that readers and writers do not block each other.
- Optimistic protocols validate at the end; lock granularity trades concurrency against overhead.

Practice Questions

1. Draw the lock compatibility matrix and explain it.
2. What are the two phases of 2PL? Why does strict 2PL avoid cascading rollbacks?
3. Compare wait-die and wound-wait with an example involving an old and a young transaction.
4. How does a wait-for graph detect deadlocks?
5. Why can MVCC let long-running reports run without slowing down updates?

Storage, Indexing and Recovery

So far we have treated tables as neat grids of data. Underneath, a database is a collection of files on disk, and disks are slow compared with memory. This final chapter looks at how data is stored, how **indexes** make searches fast, and how a DBMS **recovers** after a crash, delivering the atomicity and durability promised by ACID.

The Storage Hierarchy

Level	Speed	Size	Survives power loss?
CPU cache	Fastest	Megabytes	No
Main memory (RAM)	Very fast	Gigabytes	No
Solid-state drive (SSD)	Fast	Hundreds of GB to TB	Yes
Hard disk (HDD)	Slower	Terabytes	Yes
Tape / cloud archive	Slowest	Huge	Yes

Data lives permanently on SSD or HDD, but it can only be processed in memory. Moving data between disk and memory is the main cost of most database operations, so the DBMS tries hard to **minimise disk reads and writes**.

Blocks and the Buffer Manager

Disks transfer data in fixed-size **blocks** (or **pages**), typically 4 to 16 KB. A DBMS always reads whole blocks, even if it needs one row.

The **buffer manager** keeps recently used blocks in a region of memory called the **buffer pool**. If a requested block is already there (a *buffer hit*), no disk access is needed. When the pool is full, a replacement policy such as **LRU** (least recently used) decides which block to evict. Modified ("dirty") blocks must be written back to disk before eviction.

File Organisation

How are records arranged inside files?

- **Heap file:** records are placed wherever there is space, in no particular order. Inserts are fast, but finding a record means scanning the whole file.
- **Sequential (sorted) file:** records are kept in order of a search key. Range queries are efficient, but inserts are harder because order must be maintained.
- **Hashed file:** a hash function on a key decides which block a record goes to. Equality searches are very fast.

Indexes

An **index** is an extra data structure that helps find records quickly, just like the index at the back of a textbook. Without an index, finding the student with roll number 4521 in a million-row table requires a **full table scan**. With an index, the DBMS jumps almost directly to the right block.

```
CREATE INDEX idx_student_name ON student(name);
```

Key idea: Indexes speed up reads but slow down writes, because every INSERT, UPDATE and DELETE must also update the index. They also use extra storage. Index the columns you search and join on most often, not every column.

Types of Indexes

- **Primary (clustering) index:** built on the key by which the file itself is sorted. There can be only one, because a file can only be physically sorted one way.
- **Secondary (non-clustering) index:** built on any other column. The records are not in that order, so the index points to them wherever they are.
- **Dense index:** has an entry for *every* search-key value.

- **Sparse index:** has entries for only *some* values, typically the first key in each block. It works only on sorted files: find the largest entry not bigger than the key you want, then scan that block.
- **Multilevel index:** when an index is itself too large, build an index on the index. This idea leads directly to B+ trees.

B+ Trees

The **B+ tree** is the most widely used index structure in databases. It is a balanced search tree designed for disks.

Its properties:

- All **leaves are at the same depth**, so every search takes the same number of steps.
- Each node fits in one disk block and holds many keys, often hundreds. This high **fan-out** keeps the tree very shallow: three or four levels can index millions of records.
- **Internal nodes** hold keys and child pointers and are used only for navigation.
- **Leaf nodes** hold keys with pointers to the actual records, and each leaf links to the next leaf.
- Every node except the root is at least half full.

Searching

Start at the root. At each internal node, follow the pointer for the range containing your key, until you reach a leaf. With a height of 3, a search costs about 3 block reads, plus one to fetch the record.

Range Queries

Because leaves are linked in sorted order, a query like `cgpa BETWEEN 8 AND 9` finds the first matching leaf, then walks along the leaf chain. This makes B+ trees excellent for ranges and for `ORDER BY`.

Insertion and Deletion

To insert, find the correct leaf and add the key. If the leaf overflows, **split** it into two and push the middle key up to the parent. Splits can travel up to the root; if the root splits, the tree grows one level taller. Deletion may cause nodes to **merge** or borrow keys from siblings. These operations keep the tree balanced automatically.

Hash Indexes

A **hash index** applies a hash function to the key to find a **bucket** directly. Equality lookups (`WHERE roll_no = 4521`) take close to one block access. However, hashing scatters keys randomly, so hash indexes cannot help with range queries or sorting.

Static hashing uses a fixed number of buckets and suffers overflow chains as data grows. **Dynamic hashing** schemes such as **extendible hashing** grow the number of buckets as needed.

Need	Best choice
Equality lookups only	Hash index
Ranges, sorting, prefix search	B+ tree
Small table scanned fully anyway	No index

Failures and Recovery

Things go wrong: programs crash, power fails, disks die. The **recovery manager** makes sure that committed transactions survive (durability) and uncommitted ones leave no trace (atomicity).

Types of Failures

- **Transaction failure:** a logical error (such as dividing by zero) or a system decision (such as a deadlock victim) aborts one transaction.
- **System crash:** power loss or an operating system failure wipes out memory, but disk contents survive.
- **Disk failure:** the disk itself is damaged. Recovery needs backups or replicas.

The Log

The key tool for recovery is the **log**, a sequential file recording every change. Typical log records are:

```
<T1 start>
<T1, A, 10000, 5000>      -- T1 changed A from 10000 to 5000
                           (old, new)
<T1, B, 2000, 7000>
<T1 commit>
```

The **write-ahead logging (WAL)** rule says: a log record must be safely written to disk *before* the data change it describes is written to disk. And a transaction is committed only when its commit record is on disk.

Undo and Redo

After a crash, the recovery manager reads the log.

- Transactions with a <start> but **no** <commit> are **undone**: their changes are reversed using the old values, restoring atomicity.
- Transactions **with** a <commit> are **redone**: their changes are reapplied using the new values, in case they had not reached the disk before the crash. This ensures durability.

Undo and redo are designed to be **idempotent**: doing them twice gives the same result as doing them once, which matters if the system crashes again during recovery.

Checkpoints

Scanning the whole log after every crash would take too long. Periodically, the DBMS takes a **checkpoint**: it writes all dirty buffers to disk and records `<checkpoint>` in the log, listing the transactions that are active. Recovery then only needs to examine the log from the last checkpoint onwards.

Key idea: WAL plus checkpoints give fast normal operation *and* fast recovery. The **ARIES** algorithm, used by many commercial systems, builds on these ideas with analysis, redo and undo passes.

Shadow Paging

An alternative to logging is **shadow paging**. The DBMS keeps two page tables: the current one and a shadow copy. Changes are written to new pages. On commit, the current page table becomes the official one in a single atomic step. On abort, the shadow table is simply kept. It is simple, but it fragments data and is rarely used in large systems.

Protecting Against Disk Failure

- **Backups:** regular full and incremental copies of the database, plus archived logs, allow the database to be rebuilt up to the last committed transaction.
- **RAID:** combines several disks so that data survives one disk failing (for example, mirroring in RAID 1).

- **Replication:** keeps live copies on other servers, which can take over if the main server fails.

Summary

- Disk access dominates cost; the buffer manager caches blocks in memory.
- Files can be heap, sorted or hashed.
- Indexes speed up searches at the cost of slower writes and extra space.
- B+ trees are balanced, shallow and ideal for both equality and range queries; hash indexes are best for equality only.
- The log, the write-ahead logging rule, undo, redo and checkpoints provide atomicity and durability.
- Backups, RAID and replication protect against disk failure.

Practice Questions

1. Why does a DBMS read whole blocks rather than individual records?
2. Compare dense and sparse indexes. Why does a sparse index need a sorted file?
3. Explain why a B+ tree of height 3 can index millions of records.

4. When would you prefer a hash index to a B+ tree, and when not?
5. After a crash, the log shows T1 committed and T2 started but did not commit. What does recovery do to each, and why?